Mini-projekt: równanie Laplace’a metodą relaksacyjną w Pythonie

Plan pracy krok po kroku, zakres materiału do nauczenia, propozycja organizacji kodu oraz wskazówki do raportu / notebooka.

Cel praktyczny:	Strategia:
zbudować poprawny solver 2D dla równania Laplace’a na kwadracie, porównać wynik numeryczny z analitycznym oraz sprawdzić wpływ siatki i liczby iteracji.	najpierw bardzo prosty przypadek testowy, potem dopiero porządkowanie kodu, wykresy, analiza błędu i ewentualnie lekki refaktoring obiektowy.

Rekomendacja na start: wybierz funkcję zespoloną $f(z)=z^2$, bo jej część rzeczywista jest bardzo prosta, warunki brzegowe da się zapisać bez wysiłku, a porównanie z rozwiązaniem dokładnym jest przejrzyste.

Przykład testowy:
$f(z)=z^2$, $z=x+iy$
$u(x,y)=\mathrm{Re}(f)=x^2-y^2$
Kwadrat obliczeniowy: $[0,1] \times [0,1]$.

Dokument roboczy przygotowany jako mapa działania do implementacji własnej.

1. Co dokładnie masz zrobić

• Wybrać funkcję zmiennej zespolonej f(z), obliczyć jej część rzeczywistą u(x,y) i sprawdzić, że spełnia równanie Laplace'a.

• Wyznaczyć warunki brzegowe na bokach kwadratu o boku a.

• Napisać program w Pythonie rozwiązujący problem metodą relaksacyjną na siatce prostokątnej.

• Porównać wynik numeryczny z rozwiązaniem analitycznym dla kilku gęstości siatki i dla różnych limitów iteracji.

• Dobrać parametry tak, żeby uzyskać dokładność około 10^-4 względem rozwiązania analitycznego.

Najbezpieczniejszy wariant matematyczny

• Wybierz $f(z)=z^2$.

• Rozpisz $(x+iy)^2 = x^2 - y^2 + 2ixy$.

• Część rzeczywista: $u(x,y)=x^2-y^2$.

• Sprawdź: $u_{xx} = 2$, $u_{yy} = -2$, więc $u_{xx} + u_{yy} = 0$.

• Z tego dostajesz bardzo prosty test poprawności całego programu.

Warunki brzegowe dla kwadratu [0,1] × [0,1]

Bok	Warunek
$y = 0$	$u(x,0) = x^2$
$y = 1$	$u(x,1) = x^2 - 1$
$x = 0$	$u(0,y) = -y^2$
$x = 1$	$u(1,y) = 1 - y^2$

To są warunki Dirichleta. W tej wersji projektu nie komplikuj zadania warunkami Neumanna ani Robina. Najpierw zrób poprawny solver z ustalonym brzegiem.

2. Minimalna teoria, którą trzeba rozumieć

Temat	Co rozumieć praktycznie
Funkcja zespolona	Jak z f(z) wyciągnąć część rzeczywistą i urojoną po podstawieniu z=x+iy.
Równanie Laplace'a	Że $\Delta u = u_{xx} + u_{yy} = 0$ opisuje stan ustalony / równowagę.
Siatka różnicowa	Jak z funkcji ciągłej zrobić tablicę wartości U[i,j] w węzłach.

Druga pochodna	Centralny wzór różnicowy i jego błąd rzędu $O(h^2)$.
Iteracja relaksacyjna	Dlaczego punkt wewnętrzny zastępujesz średnią z czterech sąsiadów.
Błąd	Różnica między błędem iteracyjnym a błędem względem rozwiązania dokładnego.

Wzór kluczowy dla punktów wewnętrznych

Dla siatki kwadratowej z krokiem h obowiązuje przybliżenie:

$$U[i,j] = 1/4 \cdot (U[i+1,j] + U[i-1,j] + U[i,j+1] + U[i,j-1])$$

czyli nowa wartość w punkcie wewnętrznym jest średnią arytmetyczną z czterech sąsiadów.

To jest najważniejsze równanie w całym projekcie. Brzeg pozostaje stały, aktualizujesz wyłącznie wnętrze.

3. Plan implementacji w Pythonie

Krok A. Ustal parametry: a, N, h, tol, max_iter.

Krok B. Wygeneruj siatkę x i y oraz pustą tablicę U o wymiarze (N+1, N+1).

Krok C. Wpisz do U warunki brzegowe na czterech bokach.

Krok D. Wypełnij wnętrze zerami albo średnią z brzegów jako wartością startową.

Krok E. Uruchom iteracje Jacobi albo Gauss-Seidel dla punktów wewnętrznych.

Krok F. Po każdej iteracji licz maksymalną zmianę max|U_new - U_old|.

Krok G. Przerwij iteracje po spełnieniu tolerancji albo po przekroczeniu max_iter.

Krok H. Oblicz rozwiązanie analityczne na siatce.

Krok I. Policz błąd maksymalny i ewentualnie błąd średni.

Krok J. Powtórz obliczenia dla kilku siatek: np. N=10, 20, 40, 80, 120.

Kolejność, która oszczędza nerwy

- Najpierw wersja działająca bez wykresów.
- Potem porównanie z rozwiązaniem dokładnym.
- Dopiero na końcu estetyka kodu, wykresy i ewentualne OOP.

4. Co badać w wynikach

• Jak zmienia się liczba iteracji potrzebnych do zbieżności wraz ze wzrostem N.

• Jak zmniejsza się błąd maksymalny przy zagęszczaniu siatki.

• Czy dla zbyt małej liczby iteracji błąd pozostaje duży mimo poprawnego schematu.

• Czy dalsze zmniejszanie tolerancji iteracyjnej jeszcze coś daje, czy dominuje już błąd siatki.

Minimalny zestaw wykresów do oddania

• Mapa ciepła rozwiązania numerycznego.

• Mapa błędu |U - U_exact|.

• Przekrój, np. dla y=0.5: wynik numeryczny kontra analityczny.

• Tabela: N, h, liczba iteracji, błąd maksymalny.

Jak dojść do dokładności około 10^-4

Nie patrz tylko na liczbę iteracji. Potrzebujesz jednocześnie dwóch rzeczy: dostatecznie małego błędu iteracyjnego oraz dostatecznie gęstej siatki. Nawet idealnie zbieżna iteracja nie da bardzo dokładnego wyniku na zbyt rzadkiej siatce.

Dobra praktyka: dla każdej siatki zapisuj osobno:
1) tolerancję iteracyjną,
2) liczbę wykonanych iteracji,
3) błąd maksymalny względem rozwiązania analitycznego.

5. Czy programowanie obiektowe ma tu sens?

Tak, ale raczej jako lekki refaktoring po uruchomieniu wersji podstawowej, a nie jako punkt startowy. Sam algorytm jest prosty numerycznie i w pierwszej wersji kod proceduralny / funkcyjny bywa czytelniejszy.

Najrozsądniejszy kompromis

Wersja	Kiedy użyć	Ocena
Czysto proceduralna	Pierwsze uruchomienie i debugowanie.	Najlepsza na start
Lekko obiektowa	Gdy solver już działa i chcesz poćwiczyć klasę, pola i metody.	Polecana później
Mocno obiektowa	Gdy projekt ma wiele typów równań, brzegów i solverów.	Tu raczej przesada

Prosty wariant obiektowy, który ma sens

• Klasa ProblemConfig: przechowuje a, N, tol, max_iter, omega.

• Klasa LaplaceProblem: zawiera funkcję dokładną i metody do ustawiania warunków brzegowych.

• Klasa RelaxationSolver: ma metodę solve() oraz metody do liczenia błędu i raportowania iteracji.

> Na pierwszą wersję kodu nie wprowadzaj dziedziczenia, wzorców projektowych ani zbyt wielu klas. Jedna lub dwie klasy maksymalnie w zupełności wystarczą.

6. Checklista przed oddaniem

- Masz jasno zapisaną funkcję f(z) i część rzeczywistą u(x,y).
- Pokazałeś, że u spełnia równanie Laplace’a.
- Masz cztery warunki brzegowe na bokach kwadratu.
- Program działa dla co najmniej kilku siatek.
- Jest porównanie z rozwiązaniem analitycznym.
- Masz wykresy i tabelę błędów.
- W opisie komentujesz wpływ siatki i liczby iteracji.

Najlepszy plan pracy z asystą krok po kroku

- Najpierw napisz kod ustawiający siatkę i warunki brzegowe.
- Potem dodaj iterację relaksacyjną.
- Następnie policz błąd względem u_exact.
- Na końcu dopiero zrób wykresy i ewentualnie lekkie OOP.

> Rekomendacja końcowa: najpierw wersja prosta, potem ładna. W numeryce to zwykle oszczędza najwięcej czasu.